

AES-Enabled Industrial IoT Monitoring and Safety System

Secure Embedded Sensing, AES-128-CBC Protected MQTT Telemetry, Persistent Logging, and
Real-Time Monitoring

GitHub Project Technical Report

This report documents the standalone industrial IoT monitoring implementation based on ESP8266, sensor-level safety processing, AES-128-CBC application-layer encryption, MQTT communication, Python-based reception, SQLite storage, and a Streamlit monitoring dashboard.

1. Executive Summary

This project implements a secure industrial IoT monitoring pipeline in which an ESP8266 NodeMCU acquires environmental and proximity data, evaluates local safety conditions, encrypts telemetry using AES-128 in Cipher Block Chaining (CBC) mode, and publishes the encrypted payload over MQTT. A Python receiver decodes and decrypts the payload, extracts the recovered telemetry, stores it in SQLite, and exposes the data through a real-time Streamlit dashboard.

The demonstrated system integrates sensing, local safety logic, cryptographic protection, wireless messaging, persistent data logging, visualization, and security analytics into one end-to-end workflow. The project is intended as a reproducible engineering implementation and documentation package for GitHub.

2. Project Objectives

- Acquire temperature, humidity, gas-level, and distance measurements using an ESP8266-based sensor node.
- Evaluate predefined safety thresholds locally before transmission.
- Protect telemetry using AES-128-CBC with a 16-byte initialization vector and PKCS#7 padding.
- Transmit the encrypted application payload using MQTT.
- Receive, Base64-decode, decrypt, and reconstruct the original JSON telemetry.
- Persist recovered measurements in an SQLite database for historical analysis.
- Provide real-time monitoring through a Streamlit dashboard.
- Integrate a confidential AI-based security classification layer without exposing proprietary model implementation.

3. System Architecture

End-to-end data lifecycle: Data Acquisition → Local Processing → Safety Evaluation → Packet Formation → AES-128 Encryption → Base64 Encoding → MQTT Transmission → Reception → Base64 Decoding → Decryption → JSON Recovery → SQLite Storage → Streamlit Visualization → Security Analysis.

The ESP8266 performs sensing and local safety evaluation. AES protects the application payload before it is published. MQTT provides the messaging layer. The Python receiver performs decoding and decryption, after which the recovered data is stored and visualized.

4. Hardware Architecture

The sensor node is built around an ESP8266 NodeMCU. It interfaces with a DHT11 temperature/humidity sensor, MQ-135 gas sensor, HC-SR04 ultrasonic distance sensor, touch sensor, three status LEDs, and a buzzer.

Component	ESP8266 connection	Function
DHT11	D2 / GPIO4	Temperature and humidity
HC-SR04 TRIG	D1 / GPIO5	Ultrasonic trigger
HC-SR04 ECHO	D5 / GPIO14	Ultrasonic echo
MQ-135	A0	Gas sensor ADC level
Touch sensor	D6 / GPIO12	Local buzzer acknowledgement
Green LED	D4 / GPIO2	Normal-state indication
Yellow LED	D7 / GPIO13	Normal-state indication
Red LED	D8 / GPIO15	Danger indication

Component	ESP8266 connection	Function
Buzzer	D0 / GPIO16	Local danger alarm
Ground	Common GND	Common electrical reference

5. Sensor Acquisition and Safety Logic

The firmware periodically reads temperature, humidity, gas level, and ultrasonic distance. The measured values are evaluated locally before the telemetry packet is created. The MQ-135 value is documented as a gas sensor ADC level rather than a calibrated concentration in ppm.

The demonstrated thresholds are: temperature > 35 °C, humidity > 80%, gas ADC level > 1000, and a valid ultrasonic distance below 6 cm. A no-echo/invalid ultrasonic reading is represented as -1.0 cm and is not treated as an intrusion event.

Normal operation activates the green and yellow indicators while the red LED and buzzer remain off. A danger condition activates the red LED and buzzer while the normal indicators are disabled. The touch sensor can silence the buzzer locally without clearing the underlying danger state.

6. Telemetry Packet

Before encryption, the firmware serializes the measurements into a JSON object containing the device identifier, temperature, humidity, gas level, distance, and system status. A representative normal-state structure is:

```
{"device_id":"ESP8266_01","temperature":30.6,"humidity":49.0,"gas_level":852,"distance":57.0,"status":"NORMAL"}
```

The actual public repository should use configuration placeholders and must not expose real Wi-Fi credentials, broker credentials, private network addresses, or reusable secret keys.

7. AES-128-CBC Encryption

The telemetry payload is protected using AES-128 in Cipher Block Chaining mode. AES uses a 128-bit key and a 128-bit (16-byte) block size. The implementation generates a fresh 16-byte IV for each encrypted message, applies PKCS#7 padding, encrypts the padded plaintext block-by-block, prepends the IV to the ciphertext, and Base64-encodes the resulting binary package for MQTT transport.

CBC encryption: $C_i = E_K(P_i \oplus C_{i-1})$, with $C_0 = \text{IV}$.

CBC decryption: $P_i = D_K(C_i) \oplus C_{i-1}$.

In the implemented packet format, the binary message is arranged as [16-byte IV][ciphertext], followed by Base64 encoding. The receiver reverses these steps: Base64 decoding, IV extraction, AES-CBC decryption, PKCS#7 unpadding, and JSON reconstruction.

8. PKCS#7 Padding

AES-CBC operates on complete 16-byte blocks. PKCS#7 padding fills the final block with bytes whose value equals the number of padding bytes. For example, if 5 bytes are required, five bytes with value 0x05 are appended. If a plaintext already occupies a complete block, a complete 16-byte padding block is added.

9. Initialization Vector

A 16-byte IV is generated for each message. The IV is required for CBC decryption and is transmitted together with the ciphertext. The IV is not the secret key; its purpose is to initialize the CBC chaining process for that message.

10. Base64 Transport Encoding

AES ciphertext and IV are binary data. Base64 converts the combined binary package into an ASCII representation suitable for carrying inside the MQTT message payload. Base64 is an encoding mechanism, not an encryption mechanism.

11. MQTT Communication

The ESP8266 publishes the Base64-encoded encrypted payload to the MQTT topic `iot/sensors`. The demonstrated broker configuration uses TCP port 1883. MQTT provides the publish/subscribe messaging mechanism while AES provides application-layer confidentiality for the telemetry payload.

12. Python Receiver and Decryption

The Python receiver subscribes to the MQTT topic, obtains the encrypted Base64 payload, decodes it into bytes, extracts the IV, decrypts the ciphertext using the configured AES-128 key, removes PKCS#7 padding, and reconstructs the original JSON telemetry. The recovered fields can then be persisted for monitoring and analysis.

13. SQLite Data Storage

Recovered telemetry is stored persistently in an SQLite database. The demonstrated system accumulated more than 6,000 records during the observed run, enabling historical queries and trend visualization.

14. Streamlit Monitoring Dashboard

The Streamlit dashboard provides the monitoring and analytics layer of the system. It presents live sensor values, threshold states, actuator status, historical plots, database information, event history, and cryptographic payload inspection. The dashboard is a visualization and analysis layer; it does not perform the embedded safety decision or replace the encryption mechanism.

The documented dashboard implementation includes a modular component structure covering the header, pipeline, overview, metrics, thresholds, intrusion information, actuator state, charts, cryptographic inspector, configuration, storage table, and event history.

15. AI-Based Security Monitoring

The project includes a confidential AI-based security classification module. The public documentation intentionally describes its externally observable role without exposing proprietary implementation details.

The security-analysis layer is intended to classify telemetry/communication samples into four categories: Legitimate, Spoofed, Replayed, and Malformed. The reported project metrics are a weighted F1-score of 95.0% and an inference latency of 8.5 ms.

The AI implementation itself is confidential. Therefore, the public repository/report does not disclose proprietary model source code, architecture, training data, feature-engineering details, model parameters, or model weights. These results should be understood as the project's reported evaluation metrics rather than as a claim that the confidential implementation is independently reproducible from this repository.

16. Security Considerations

AES-128-CBC is used here as an application-layer confidentiality mechanism. CBC by itself does not provide authenticated integrity. Accordingly, the report does not claim that AES-CBC alone detects packet manipulation or authenticates the sender. MQTT remains the communication transport, while cryptographic protection is applied to the application payload.

For public GitHub publication, all secrets must be removed from source files. Wi-Fi passwords, AES keys, MQTT credentials, private IP addresses, database files containing sensitive data, and other credentials should be excluded using environment variables, example configuration files, or placeholders.

17. Demonstrated Results

Representative normal-operation values observed in the documented prototype include approximately 30.6 °C temperature, 48–49% humidity, gas ADC levels around 845–852, and distance around 57 cm. These values are examples from the demonstrated run and are not universal operating limits.

Category	Documented result
Normal temperature example	≈ 30.6 °C
Normal humidity example	≈ 48–49 %
Normal gas ADC example	≈ 845–852
Normal distance example	≈ 57 cm
Safety temperature threshold	> 35 °C
Safety humidity threshold	> 80 %
Safety gas threshold	> 1000 ADC level
Intrusion distance threshold	< 6 cm
Database history	6,000+ demonstrated records

Category	Documented result
AI weighted F1-score	95.0% (reported)
AI inference latency	8.5 ms (reported)

18. Reproduction Workflow

- Install Arduino IDE and add the ESP8266 board package.
- Select NodeMCU 1.0 (ESP-12E Module) and the correct COM port.
- Install the firmware dependencies used by the source code, including ESP8266WiFi, PubSubClient, DHT, Crypto, and AES libraries as applicable to the project environment.
- Configure Wi-Fi and MQTT settings using local/private configuration values rather than committing secrets.
- Connect the sensor node according to the documented pin table and verify the common ground.
- Upload the ESP8266 firmware and monitor the serial output at 115200 baud.
- Run the Mosquitto MQTT broker and verify publication to iot/sensors.
- Run the Python receiver to decode, decrypt, and store telemetry.
- Launch the Streamlit dashboard and verify live values, historical records, and cryptographic inspection.
- Keep confidential AI implementation files outside the public repository unless explicitly authorized for release.

19. Recommended Public Repository Structure

AES-Enabled-Industrial-IoT-Monitoring-and-Safety-System/

```

■■■■ README.md
■■■■ LICENSE
■■■■ .gitignore
■■■■ requirements.txt
■■■■ firmware/
■ ■■■■ ESP8266_Secure_IoT/
■■■■ receiver/
■■■■ dashboard/
■■■■ hardware/
■■■■ documentation/
■ ■■■■ architecture/
■ ■■■■ Project_Report.pdf
■■■■ screenshots/
■■■■ samples/

```

20. GitHub Publication and Confidentiality Checklist

- Remove all real Wi-Fi SSIDs/passwords and MQTT credentials from public source.
- Remove or replace any reusable AES secret key with a configuration placeholder.
- Do not commit private IP addresses when they are environment-specific.
- Do not commit the production SQLite database if it contains sensitive or unnecessary records.
- Use sample telemetry and sanitized screenshots for public demonstration.
- Keep the confidential AI model, weights, training dataset, proprietary features, and internal training code outside the public repository.
- Document setup instructions using example configuration values.
- Include the project report and architecture diagrams in the documentation directory.

21. Engineering Summary

The AES-Enabled Industrial IoT Monitoring and Safety System demonstrates an end-to-end embedded IoT workflow in which physical sensing, local safety processing, cryptographic protection, MQTT communication, server-side decryption, persistent storage, and real-time visualization operate as a connected system. The ESP8266 provides the sensing and local decision layer, AES-128-CBC protects the telemetry payload, MQTT carries the encrypted application data, Python reconstructs the telemetry, SQLite provides persistence, and Streamlit provides the monitoring interface. A separate confidential AI security-analysis layer extends the platform with reported classification performance while keeping proprietary implementation details private.

22. Conclusion

The project provides a complete engineering implementation of secure industrial IoT telemetry from sensor acquisition to dashboard visualization. Its main contribution is the integration of embedded sensing, local safety logic, AES-128-CBC protected application data, MQTT messaging, Python-based reception, SQLite persistence, and real-time Streamlit monitoring into a single reproducible workflow. The public documentation deliberately separates reproducible system components from confidential AI implementation details and excludes sensitive deployment credentials.

Project Title: AES-Enabled Industrial IoT Monitoring and Safety System

Primary Technologies: ESP8266, DHT11, MQ-135, HC-SR04, MQTT, AES-128-CBC, Base64, Python, SQLite, Streamlit

Documentation Type: GitHub Project Technical Report